🚀 FinTrackr — System Design & Documentation

🧭 1. SYSTEM FLOW (HIGH-LEVEL)

Client (Postman / UI)
↓
JWT Authentication
↓
Spring Security (RBAC)
↓
Controller Layer
↓
Service Layer (Business Logic)
↓
Repository Layer
↓
MongoDB Atlas (Cloud Database)

💡 2. INTRODUCTION

🔹 Project Overview

FinTrackr is a secure, role-based backend system designed for managing and analyzing company-level financial data.
It enables organizations to:

- Track income and expenses
- Generate real-time financial insights
- Enforce role-based access control
- Maintain centralized financial records

🎯 Problem Statement

Organizations often face:

- Scattered financial data
- Lack of centralized tracking
- No real-time analytics

👉 Solution
FinTrackr centralizes financial data → processes it → generates actionable insights.

🧠 3. SYSTEM TYPE

- Centralized Company Finance Backend
- Not a personal expense tracker
- Transactions represent organization-wide data (not user-specific)

🧱 4. SYSTEM ARCHITECTURE

🔹 Layered Architecture

Layer	Responsibility
Controller	Handles HTTP requests/responses
Service	Business logic, validation, processing
Repository	Database interaction
Database	Stores users & transactions

🔹 Why Layered Architecture?

- Separation of concerns
- Easier debugging & testing
- Scalable and maintainable design

🔄 5. REQUEST LIFECYCLE (END-TO-END)

Client sends request with JWT
↓
JWT Filter validates token
↓
Extract role from token
↓
Spring Security checks permissions
↓
Controller → Service → Repository
↓
MongoDB operation
↓
Response returned

🔐 6. AUTHENTICATION FLOW

Register
↓
OTP Generated
↓
Verify OTP
↓
User Activated
↓
Login
↓
JWT Token Generated

🔹 Key Points

- Secure identity verification via OTP
- Stateless authentication using JWT

🔒 7. AUTHORIZATION (RBAC)

👥 Roles & Access

Role	Access
ADMIN	Full access (CRUD + dashboard)
ANALYST	Dashboard + filtering
VIEWER	Read-only dashboard

🔄 Authorization Flow

Request with JWT
↓
JWT Filter extracts role
↓
@PreAuthorize check
↓
Allowed → proceed
Denied → 403 Forbidden

🗄️ 8. DATA MODELS

👤 User

- id
- email (unique)
- password (hashed using BCrypt)
- role (ADMIN / ANALYST / VIEWER)
- active
- otp

💰 Transaction

- id
- amount
- type (INCOME / EXPENSE)
- category
- date
- notes

🔹 MongoDB Collections

- users
- transactions

🔥 9. CORE FEATURES

🟢 A) Create Transaction

POST /transactions
👉 Admin creates financial records

📊 B) Dashboard APIs

💰 Total Income
GET /transactions/income

💸 Total Expense
GET /transactions/expense

📊 Net Balance
GET /transactions/balance
👉 Net = Income - Expense

📦 Category Breakdown
GET /transactions/category-breakdown

- Groups by category

- Calculates totals

📈 Trends (Profit/Loss)
GET /transactions/trends

- Groups by date
- Profit = Income - Expense

📜 Recent Activity

- Latest 5 transactions
- Sorted by date (DESC)

🔄 C) Update Transaction

PUT /transactions/{id}

❌ D) Delete Transaction

DELETE /transactions/{id}

🔍 E) Filtering

GET /transactions?type=EXPENSE&category=MARKETING

🔄 10. DATA FLOW

🟢 Create Transaction
Admin → Controller → Service → Repository → MongoDB

📊 Dashboard Processing
Fetch transactions → Aggregate → Compute → Return response

📈 Trends Processing
Group by date → Calculate profit/loss → Return structured data

⚡ 11. SYSTEM BEHAVIOR

- Real-time updates (no caching)
- Every API call reflects latest data
- Stateless system (no session storage)

🧠 12. KEY DESIGN DECISIONS

✅ No userId in transactions
→ Organization-level system
✅ JWT Authentication
→ Stateless, scalable, secure
✅ RBAC
→ Controlled access
✅ Category as String
→ Flexible schema
✅ Type as ENUM
→ Strict validation

⚠️ 13. CONSTRAINTS

- Unique email enforced
- No plaintext passwords
- Admin creation requires secret
- No direct DB access from controller

⚠️ 14. LIMITATIONS

- No caching (performance can degrade at scale)
- No pagination in APIs
- No audit logging
- No frontend (backend-only system)

⚙️ 15. TECH STACK

Backend
- Java
- Spring Boot

Database
- MongoDB Atlas

Security
- Spring Security
- JWT
- BCrypt

DevOps
- Docker (Multi-stage build)
- Render (Cloud deployment)

🚀 16. DEPLOYMENT

- Dockerized using multi-stage build
- Deployed on Render
- Connected to MongoDB Atlas

🏁 17. CONCLUSION

FinTrackr demonstrates:

- Clean layered architecture
- Secure authentication & authorization
- Real-time financial analytics
- Scalable backend design

🎯 18. SUMMARY

- FinTrackr is a centralized finance backend system where users authenticate using JWT after OTP verification.
- Each request is validated via a JWT filter and role-based access is enforced using Spring Security.
- The system follows a layered architecture (Controller → Service → Repository → MongoDB).
- Admin users manage transactions, while dashboard APIs provide real-time insights like income, expenses, trends, and category breakdown.
- The system is stateless, secure, and designed for scalable financial analytics.

👩‍💻 AUTHOR

Anupriya Poddar
anupriyapoddar.work@gmail.com